Parallel PageRank with MPI+OpenMP and MPI+CUDA

DSA5208 Project 3

Ma Xiaoxiao A0259173X
Ye Sifan A0262394Y
Chen Yuda A0353182H

November 2026

Abstract

We implement the PageRank power iteration for sparse directed graphs as a serial C++ program, an MPI+OpenMP program, a CUDA program for one GPU and an MPI+CUDA program for several GPUs. The parallel programs split the in-edge matrix of the graph into contiguous row blocks with equal numbers of edges. Each MPI rank owns one block and, in every iteration, receives from the other ranks exactly the vector entries that its edges refer to. All programs are measured on the NUS Vanda cluster with graphs of up to 65.6 million vertices and 3.6 billion edges, on up to 720 CPU cores and two NVIDIA A40 GPUs. With one MPI rank per NUMA domain and 36 OpenMP threads per rank, one iteration on com-friendster takes 139 ms on 576 cores instead of 25.3 s on one core, a speedup of 183. With one rank per core, the same program is fastest on 144 cores: beyond that, the exchange between ranks consists of up to $P(P-1)$ messages per iteration for P ranks, and over TCP its cost follows the number of messages, not the amount of data. One A40 GPU is as fast as a 72-core node on soc-LiveJournal1, and two GPUs run com-friendster 1.68 times faster than one. Every parallel variant reproduces the serial PageRank vector to rounding error.

1 Introduction

PageRank [1] assigns to every vertex of a directed graph the long-run probability that a random walk visits it. It was introduced to rank web pages for search and is now a standard measure of importance in any network: it ranks accounts and items in social networks and recommender systems, papers and journals in citation networks, and genes and proteins in biological networks [2]. The graphs in these applications have millions to billions of edges and change continuously, so the ranks are recomputed again and again, and the time of one iteration decides how large a graph can be ranked and how fresh the ranks are.

We chose PageRank for this project for that reason and because it is a demanding case for parallelisation. It is computed by a power iteration whose step is a sparse matrix–vector product with the adjacency matrix of the graph. The arithmetic is simple, one addition per edge, so the run time is set by how fast vector entries can be fetched from memory and, in a distributed program, from other processes. Which entries a process needs is decided by the graph, and in the power-law graphs studied here almost every process needs entries from almost every other process. PageRank is therefore not embarrassingly parallel, and its scaling depends on how the graph is partitioned and how the processes communicate.

Section 2 describes the algorithm and its serial implementation, and Section 3 the design of the MPI+OpenMP, CUDA and MPI+CUDA versions. Section 4 introduces the graphs and the measurement setup, and Section 5 the validation. The results follow: serial run time against problem size (Section 6), strong and weak scaling of the MPI program and the role of communication (Section 7), and the two MPI+X combinations (Section 8). Section 9 discusses the data sets and the design as a whole, and what the results mean for applications of PageRank. The source code, the job scripts and the raw results are available at `https://github.com/yespiggy/Project3/tree/submission`.

2 Algorithm

2.1 PageRank

Let $G = (V, E)$ be a directed graph with $N = |V|$ vertices and $M = |E|$ edges. A random surfer at vertex u follows one of the outdeg(u) out-edges of u with probability d, and jumps to a vertex chosen uniformly at random with probability $1 - d$; we use the usual damping factor $d = 0.85$. A vertex without out-edges is called *dangling*, and from a dangling vertex the surfer always jumps. The PageRank vector r is the stationary distribution of this walk. It is the limit of the iteration

$$r_v^{(k+1)} = \frac{1-d}{N} + \frac{d}{N}\sum_{u \in D} r_u^{(k)} + d \sum_{(u,v)\in E} \frac{r_u^{(k)}}{\text{outdeg}(u)}, \qquad r_v^{(0)} = \frac{1}{N}, \tag{1}$$

where D is the set of dangling vertices. The three terms are the random jump, the rank of the dangling vertices spread evenly over all vertices, and the rank arriving along the in-edges of v. The iteration stops when the residual $\rho_k = \|r^{(k+1)} - r^{(k)}\|_1$ falls below a tolerance, or after a fixed number of iterations.

Iteration (1) applies a column-stochastic matrix scaled by d, so $\rho_{k+1} \le d\,\rho_k$: the residual shrinks by at least 15% per iteration on any graph. Since $\rho_0 \le 2$, a tolerance of 10^{-10} is reached after at most 147 iterations.

2.2 Serial implementation

The graph is stored as the sparse $N \times N$ matrix A with $A_{vu} = 1$ for every edge $u \to v$, in compressed sparse row (CSR) format: an offset array `off` with $N + 1$ entries and an index array `src` with M entries, such that `src[off[v]]`, …, `src[off[v+1]-1]` are the sources of the in-edges of v. We call this the *in-edge CSR*. A third array holds the out-degrees. With $x_u = r_u/\text{outdeg}(u)$, and $x_u = 0$ for dangling u, the last sum in (1) is the sparse matrix–vector product (SpMV) $y = Ax$.

This is the *pull* formulation: every vertex reads the contributions of its in-neighbours and writes only its own entry of r. Rows can be processed in any order and by different threads without atomic updates. In the alternative push formulation, every vertex adds its contribution to the entries of its out-neighbours, and concurrent writes to the same entry must be protected.

One iteration of the serial program (`src/serial/pagerank_serial.cpp`) consists of three steps.

1. **Contributions.** Compute x_u for every vertex and accumulate the dangling mass $\delta = \sum_{u\in D} r_u$.
2. **SpMV and update.** For every vertex v, compute $s_v = \sum_{e=\texttt{off}[v]}^{\texttt{off}[v+1]-1} x_{\texttt{src}[e]}$ and $r'_v = (1-d)/N + d\,\delta/N + d\,s_v$, and accumulate the residual $\sum_v |r'_v - r_v|$.
3. **Swap** r and r' and test the residual.

One iteration takes $\Theta(N + M)$ time and the program $\Theta(N + M)$ memory. The arithmetic is one addition per edge and about five operations per vertex. The memory traffic is larger: per edge, a 4-byte index is read in sequence and an 8-byte entry of x is read at the position given by that index. In a graph whose vertex numbering has no locality, consecutive reads of x fall on unrelated cache lines, and each of them loads a 64-byte line to use 8 bytes. The arithmetic intensity, the number of floating-point operations per byte moved, is about 1/12. The performance of every version in this report is therefore governed by memory and network traffic, not by arithmetic.

3 Parallel design

3.1 Partitioning

The parallel programs distribute the rows of A, that is, the vertices together with their in-edges, over P MPI ranks (processes). The partition is one-dimensional and contiguous: rank i *owns* the vertices $[\ell_i, h_i)$, their entries of r and x, and their part of the index array (Figure 1). The boundaries are chosen in one of two ways:

- *vertex partition*: every rank owns N/P vertices;
- *edge partition*: a binary search in the offset array places the boundaries such that every rank owns M/P in-edges, to within one row.

The work of a rank in the SpMV is proportional to its number of in-edges m_i. We measure how evenly the work is spread by the *load imbalance* $\max_i m_i/(M/P)$, which is 1 for a perfect partition. Rank 0 reads the offset array, computes the boundaries and broadcasts them; every rank then reads its own slice of the graph file, so no rank ever holds the whole graph.

3.2 Communication

To process its rows, rank i needs x_u for every source u of its in-edges. Sources inside $[\ell_i, h_i)$ are local. The others are owned by other ranks; we call them the *ghost vertices* of rank i. They play the role of the halo cells of a stencil code, except that the graph, not a regular grid, decides which they are. Two ways of obtaining the ghost values are implemented.

- *Allgather.* Every rank receives the complete vector x with `MPI_Allgatherv`, that is $8(N - n_i)$ bytes per iteration for a rank that owns n_i vertices, whatever it needs.
- *Ghost exchange.* Every rank receives exactly its ghost values. During setup, each rank collects the distinct ghost vertices of its in-edges in sorted order and sends every owner the list of vertices it needs from it (`MPI_Alltoallv`). It then renumbers its column indices into the local index space [owned | ghosts], so that the SpMV loop addresses a single local array (Figure 1). The ranks that exchange data are joined in a neighbourhood communicator (`MPI_Dist_graph_create_adjacent`). In every iteration, a rank packs the requested entries into a send buffer, and one call of `MPI_Neighbor_alltoallv` delivers all ghost values.

Every iteration also needs two global sums: the dangling mass before the SpMV and the residual after it. Each is an `MPI_Allreduce` of a single number. One iteration on rank i is therefore

1. compute x_u for the owned vertices and the local dangling mass;
2. pack and exchange the ghost values (or allgather x);
3. `MPI_Allreduce` of the dangling mass;
4. SpMV over the owned rows, new r, local residual;
5. `MPI_Allreduce` of the residual.

Steps 1 and 4 and the packing are computation; the three MPI calls are communication. The exchange plan is built once and reused in every iteration.

3.3 MPI+OpenMP

Inside a rank, step 1, the packing and step 4 are loops over the owned vertices, and OpenMP shares them among the threads of the rank (`src/cpu/pagerank_mpi_omp.cpp`). Step 1 and the packing

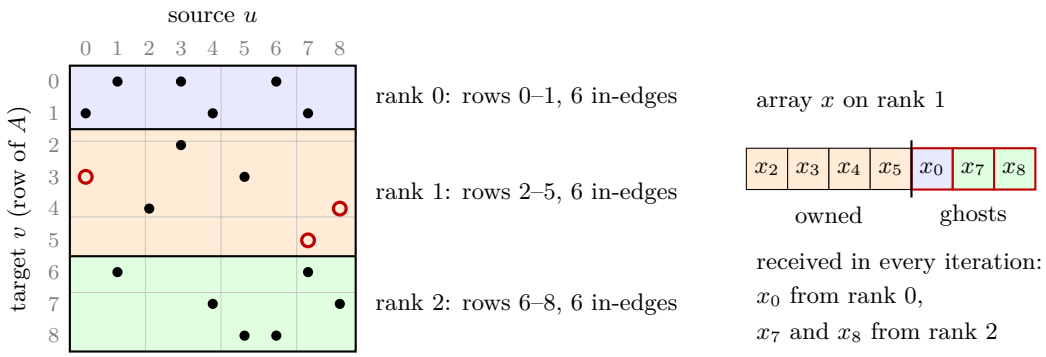


Figure 1. Edge partition of a graph with 9 vertices and 18 edges over three ranks. Each dot is an edge $u \to v$, stored in row v. The blocks hold equal numbers of in-edges and therefore different numbers of rows. Rank 1 owns $x_2, \ldots, x_5$; the three ringed edges refer to vertices owned by other ranks, its ghost vertices.

use a static schedule, because every vertex costs the same. The SpMV loop uses a dynamic schedule with chunks of 512 rows (`schedule(dynamic,512)`): the cost of a row is its in-degree, which varies by three orders of magnitude, and with a dynamic schedule a thread that meets long rows does not hold up the others. The dangling mass and the residual are OpenMP reductions. MPI is initialised with `MPI_THREAD_FUNNELED`: only the master thread calls MPI, between the parallel loops. The threads of a rank share its arrays, including the ghost values, so a value that several threads need is received once.

The same executable runs with any number of ranks and threads. Two layouts are used throughout the report.

- *Pure MPI*: one single-threaded rank per core, p ranks on p cores. This is the MPI-only program.
- *Hybrid*: one rank per NUMA domain with 36 threads. A NUMA domain is one CPU socket with the memory attached to it; a Vanda node has two domains of 36 cores. Up to 36 cores the hybrid layout is a single rank with p threads and involves no communication.

In both layouts every thread is pinned to its own core, and "p cores" always means that ranks $\times$ threads $= p$.

3.4 CUDA and MPI+CUDA

The single-GPU program (`src/gpu/pagerank_cuda.cu`) keeps the CSR arrays and the vectors in GPU memory for the whole run. One iteration launches a kernel for the contributions and a kernel for the SpMV and update. The dangling mass and the residual are summed on the GPU by a reduction in every thread block (`BlockReduce` of the CUB primitives shipped with CUDA) followed by one atomic addition per block, and only the residual is copied to the host. Four SpMV kernels are provided:

- *one thread per row*: a GPU thread loops over the in-edges of one vertex;
- *one warp per row*: the 32 threads of a warp (the group of threads that the GPU executes in lockstep) share the in-edges of one vertex and combine their partial sums with a shuffle reduction;
- *degree buckets*: the vertices are sorted once into three lists by in-degree and processed by one thread (at most 16 in-edges), one warp (17 to 1024) or one block of 256 threads (more than 1024);
- `cusparseSpMV` from NVIDIA's cuSPARSE library, as a reference point for our own kernels. No BLAS, LAPACK or sparse-matrix library is used in any other kernel or program.

The MPI+CUDA program (`src/gpu/pagerank_mpi_cuda.cu`) runs one rank per GPU and reuses the edge partition and the ghost exchange of the CPU program. In every iteration, the GPU computes the contributions and packs the send buffer; the buffer is copied to pinned host memory; the host calls `MPI_Neighbor_alltoallv` and `MPI_Allreduce`; the received ghost values are copied to the GPU; and the GPU runs the SpMV with the thread-per-row or the degree-bucket kernel. The offsets are 64-bit integers, so that a graph with more than 2^{31} edges fits on one or several GPUs; the single-GPU program uses the 32-bit offsets that cuSPARSE requires. The kernels of the two programs differ in one more detail: the single-GPU kernels read δ from GPU memory and compute the constant term of (1) for every row, whereas the MPI+CUDA program computes it once on the host after the reduction and passes it to the kernel.

Table 1 collects the terms and symbols used in the following sections.

Table 1. Terms and notation.

Term	Meaning
N, M, P, p	Vertices, edges, MPI ranks, CPU cores (ranks × threads).
In-edge CSR	Sparse matrix A with one row per vertex that lists the sources of its in-edges.
Rank, thread, core	An MPI *rank* is one process. It runs one or more OpenMP *threads*, each pinned to one CPU *core*.
Pure MPI, hybrid	The two layouts of the MPI+OpenMP program: one rank per core, or one rank per NUMA domain with 36 threads.
Owned, ghost vertex	A rank owns the vertices of its row block. A ghost vertex of a rank is owned by another rank and is the source of one of the rank's in-edges.
Ghost exchange	Delivery of the ghost values with `MPI_Neighbor_alltoallv`; the alternative is the allgather of the whole vector x.
Load imbalance	Largest number of in-edges of a rank divided by the average M/P.
T_p	Time per iteration on p cores: wall time of the iteration loop, maximum over ranks, divided by the number of iterations.
Speedup, efficiency	$S_p = T_1/T_p$ and $E_p = S_p/p$ on a fixed graph (strong scaling), where T_1 is the MPI+OpenMP program on one core.
Weak scaling	The graph grows with the core count such that the number of edges per core, M/p after duplicate removal, stays constant. Weak-scaling efficiency: T_1/T_p.
t_{comp}, t_{comm}	Time per iteration in local computation and in MPI calls. Each is the maximum over ranks, so the two need not add up to T_p. t_{comm} includes waiting for other ranks inside an MPI call.
t_{wait}	In runs with the option `--split`, a barrier precedes every MPI call. The time in the barriers is t_{wait}, the wait for the slowest rank, and t_{comm} is then the data movement alone.
Effective bandwidth	$B = (12M + 44N)/T_p$: the bytes one iteration must read and write (a 4-byte index and an 8-byte value per edge; offset, out-degree, r, x and r' per vertex) divided by its time. The single-GPU program has 4-byte offsets and counts $40N$.
STREAM triad	The loop $a_i = b_i + 3c_i$ over arrays much larger than the cache [6]; it measures the memory bandwidth for sequential access.
R-MAT, edge factor	Random graph generator with a power-law degree distribution [4]; the edge factor is the number of generated edges per vertex.

4 Data sets and experimental setup

Real graphs. We use three graphs from the SNAP collection [3]. *web-Google* is a graph of web pages and the hyperlinks between them; it is small and serves for validation and for the analysis of the PageRank values. *soc-LiveJournal1* is the directed friendship graph of the LiveJournal community. *com-friendster* is the friendship graph of the Friendster network. It is undirected, so every friendship is stored as two directed edges, every vertex has the same in- and out-degree, and no vertex is dangling. `tools/preprocess.cpp` reads the SNAP edge lists, renumbers the vertices to $0, \ldots, N-1$ in the order of their original identifiers, removes self-loops and duplicate edges, and writes the in-edge CSR as a binary file with 64-bit offsets and 32-bit indices. soc-LiveJournal1 contained 518,382 self-loops; no graph contained duplicate edges.

Table 2. Graphs after preprocessing. The density M/N^2 is the fraction of nonzero entries of A. For the R-MAT family the smallest and the largest graph are listed.

graph	N	M	M/N	density	dangling	max in-deg.	max out-deg.
web-Google	875,713	5,105,039	5.8	6.7×10^{-6}	15.6%	6,326	456
soc-LiveJournal1	4,847,571	68,475,391	14.1	2.9×10^{-6}	11.4%	13,905	20,292
com-friendster	65,608,366	3,612,134,270	55.1	8.4×10^{-7}	0%	5,214	5,214
`rmat_p1`	131,072	1,942,879	14.8	1.1×10^{-4}	41%	–	–
`rmat_p720`	94,371,840	1,495,592,066	15.8	1.7×10^{-7}	59%	–	–

Synthetic graphs. `tools/rmat_gen.cpp` implements the R-MAT generator [4] with the quadrant probabilities 0.57, 0.19, 0.19 and 0.05 of the Graph500 benchmark [5]. For every edge it descends $\lceil \log_2 N \rceil$ levels of the adjacency matrix; at each level it picks one of the four quadrants with these probabilities, which fixes one bit of the source and one bit of the target. Edges with an endpoint $\geq N$ are drawn again. The vertex labels are then permuted at random, so that the high-degree vertices, which R-MAT places at small identifiers, are spread over the whole range. Self-loops and duplicate edges are removed. The edges are generated by 32 OpenMP threads, each with its own `mt19937_64` generator seeded from the graph seed and the thread number, so a graph is reproducible from its seed and the number of threads. Two families are used:

- `rmat_p`k has $N = 131{,}072\,k$ vertices and edge factor 16, for $k = 1$, 2, 4, 8, 16, 36, 72, 144, 288, 432, 576 and 720. Run on k cores, these graphs have about 2 million edges per core. They are used for weak scaling and for the serial study.
- `rmat_n20_ef`f has $N = 2^{20}$ vertices and edge factor $f = 4$, 8, 16, 32 and 64. They are used for the serial study.

Sparsity. All graphs are very sparse: fewer than 7 of every million entries of A are nonzero for the real graphs, and the density falls as a graph grows, because the average degree stays bounded while N^2 grows (Table 2). Sparsity shapes the computation in three ways. First, the work per iteration is $\Theta(N + M)$ and not $\Theta(N^2)$, and it consists of irregular reads of x rather than of arithmetic. Second, the nonzeros are spread unevenly: the largest in-degree is hundreds to thousands of times the average, so blocks with equal numbers of vertices contain very different numbers of edges. Third, the nonzeros of a row block are scattered over all columns, so a rank needs values from most other ranks, and its number of ghost vertices grows with the size of the whole graph, not only with the size of its own block. The second point motivates the edge partition (Section 7.4) and the third limits the scaling (Section 7.3).

Platform. All measurements were made on the NUS Vanda cluster. A CPU node has two Intel Xeon Platinum 8452Y processors, i.e. two NUMA domains of 36 cores, each with 2 MB of L2 cache per core and 67.5 MB of shared L3 cache, and 503 GB of memory. A GPU node has the same processors and two NVIDIA A40 GPUs with 48 GB of memory each. The programs were compiled with GCC 13.1 (`-O3 -march=x86-64-v3`), OpenMPI 4.1.8 and CUDA 12.4 (`nvcc -O3`). CPU jobs requested whole 72-core nodes, up to 10 nodes (720 cores). GPU jobs used one or two GPUs, either on one node or one on each of two nodes. Ranks were placed and bound with `--map-by ppr:`k`:node:PE=`t `--bind-to core` for k ranks per node with t threads, and threads were pinned with `OMP_PROC_BIND=close` and `OMP_PLACES=cores`.

MPI messages within a node use OpenMPI's shared-memory transport, in which a message is copied into a shared buffer and out again. Messages between nodes use TCP. The OpenMPI installation on Vanda has no UCX support, and its InfiniBand-verbs transport cannot open connections on the Ethernet-based RDMA adapters of the nodes; we tested both of its connection schemes in a two-node job (`jobs/final/f5_ibprobe.pbs`).

Measurement. Performance runs execute a fixed number of iterations with the tolerance set to zero: 50 iterations for soc-LiveJournal1 and the R-MAT graphs, and 10 for com-friendster, where one iteration takes 25 s on one core. The parallel programs run one warm-up repetition and then 5 timed repetitions (3 for the CPU runs on com-friendster) inside the same `mpirun`, each started after a barrier. The serial program runs 3 repetitions of 10 iterations, is not pinned and has no warm-up. The time per iteration T_p of a configuration is the median over its repetitions. Loading the graph, building the partition and the exchange plan, and copying the graph to the GPU are not part of T_p. All figures, tables and numbers in this report are produced from the raw result files by `tools/make_report.py`.

5 Validation

The implementations are checked at three levels.

Table 3. Difference between the PageRank vector of each parallel variant and that of the serial program. web-Google is run to convergence (residual below 10^{-10}, 118 iterations in every variant); the MPI+OpenMP variants use 4 ranks with 4 threads each. soc-LiveJournal1 is run for exactly 50 iterations. L_1 and L_∞ are the norms of the difference; the last column says whether the 100 highest-ranked vertices appear in the same order.

graph	variant	L_1	L_∞	same top 100
web-Google	MPI+OpenMP, vertex partition, allgather	3.2×10^{-15}	4.8×10^{-18}	yes
web-Google	MPI+OpenMP, vertex partition, ghost	3.2×10^{-15}	4.8×10^{-18}	yes
web-Google	MPI+OpenMP, edge partition, allgather	3.3×10^{-15}	5.0×10^{-18}	yes
web-Google	MPI+OpenMP, edge partition, ghost	3.3×10^{-15}	5.0×10^{-18}	yes
web-Google	CUDA, one thread per row	3.0×10^{-15}	4.4×10^{-18}	yes
web-Google	CUDA, one warp per row	3.4×10^{-15}	8.9×10^{-18}	yes
web-Google	CUDA, degree buckets	3.4×10^{-15}	9.1×10^{-18}	yes
web-Google	CUDA, `cusparseSpMV`	3.4×10^{-15}	9.0×10^{-18}	yes
web-Google	MPI+CUDA, 1 GPU	3.5×10^{-15}	9.1×10^{-18}	yes
web-Google	MPI+CUDA, 2 GPUs	3.6×10^{-15}	9.2×10^{-18}	yes
soc-LiveJournal1	pure MPI, 144 ranks, 2 nodes	1.6×10^{-14}	4.4×10^{-18}	yes
soc-LiveJournal1	pure MPI, 288 ranks, 4 nodes	1.6×10^{-14}	4.4×10^{-18}	yes
soc-LiveJournal1	pure MPI, 432 ranks, 6 nodes	1.6×10^{-14}	4.4×10^{-18}	yes
soc-LiveJournal1	pure MPI, 576 ranks, 8 nodes	1.6×10^{-14}	4.4×10^{-18}	yes
soc-LiveJournal1	pure MPI, 720 ranks, 10 nodes	1.7×10^{-14}	4.4×10^{-18}	yes

Small graphs against NetworkX. `tools/smoke.py` builds four small graphs that contain the special cases of the algorithm: a directed cycle, a graph with dangling and isolated vertices, a graph without any edge (all rank is dangling mass), and a graph with a single edge. It compares the serial program and the MPI program with the `pagerank` function of NetworkX. The MPI program is run with 1, 2 and 4 ranks, both partitions and both exchange methods; with 4 ranks on these graphs some ranks own no vertex or no edge. All 52 comparisons agree to $L_1 < 10^{-10}$, and the ranks sum to 1 within 10^{-12}.

Every variant against the serial program. On web-Google, run to convergence, the serial vector agrees with NetworkX to $L_1 = 1.3 \times 10^{-10}$, with the 100 highest-ranked vertices in the same order. Every parallel variant is then compared with the serial vector (Table 3): the four combinations of partition and exchange method of the MPI+OpenMP program, the four kernels of the CUDA program, and the MPI+CUDA program on one and on two GPUs. All of them converge in the same 118 iterations, differ from the serial vector by at most $L_1 = 3.6 \times 10^{-15}$, and give the same order of the top 100 vertices. Differences of this size are rounding errors: the variants add the same numbers in different orders.

At scale. The configurations with the most ranks and the most complex exchange pattern are tested directly. soc-LiveJournal1 is run for exactly 50 iterations with pure MPI on 144 to 720 ranks (2 to 10 nodes), the vectors are written to disk and compared with the serial vector after 50 iterations (`tools/compare_vec.py`). The difference is at most $L_1 = 1.7 \times 10^{-14}$.

Every timed run. Each timed repetition writes its final residual and the sum of its rank vector to the result file. Over all 719 repetitions the sum differs from 1 by at most 6.7×10^{-10}, and all runs of the same graph and iteration count report the same residual to the four digits that are recorded, whatever the program, the layout and the number of nodes.

6 Serial run time and problem size

The problem size is varied in two ways with R-MAT graphs: the number of edges at a fixed number of vertices (`rmat_n20_ef`f, $N = 2^{20}$, M from 4.1 to 60.7 million), and vertices and edges together at a fixed ratio $M/N \approx 15.6$ (`rmat_p`k, N from 0.13 to 94 million and M from 1.9 million to 1.5 billion). The three real graphs are measured as well (Figure 2, Table 4).

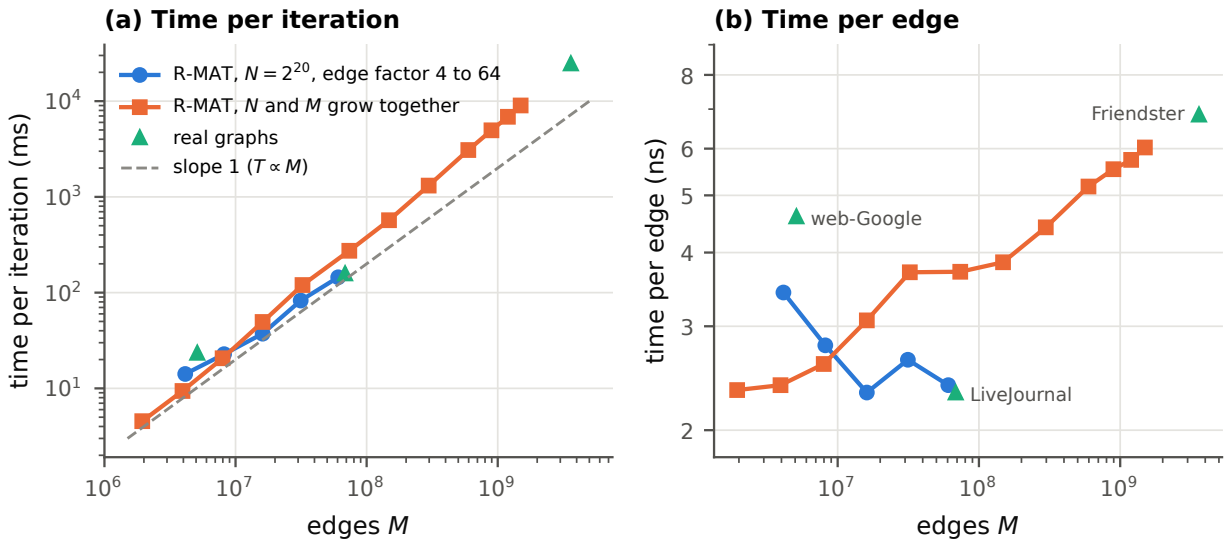


Figure 2. Serial program on one core. (a) Time per iteration against the number of edges. (b) The same time divided by the number of edges.

Table 4. Serial time per iteration. $8N$ is the size of the vector x.

family	graph	N (10^6)	M (10^6)	M/N	$8N$ (MB)	ms / iteration	ns / edge
N fixed	`rmat_n20_ef4`	1.05	4.1	3.9	8	14.1	3.42
N fixed	`rmat_n20_ef8`	1.05	8.2	7.8	8	22.8	2.79
N fixed	`rmat_n20_ef16`	1.05	16.1	15.3	8	37.3	2.32
N fixed	`rmat_n20_ef32`	1.05	31.4	29.9	8	82.7	2.63
N fixed	`rmat_n20_ef64`	1.05	60.7	57.9	8	145	2.38
N, M grow	`rmat_p1`	0.13	1.9	14.8	1	4.54	2.34
N, M grow	`rmat_p2`	0.26	3.9	15.0	2	9.39	2.38
N, M grow	`rmat_p4`	0.52	8.0	15.2	4	20.6	2.59
N, M grow	`rmat_p8`	1.05	16.1	15.3	8	49.4	3.07
N, M grow	`rmat_p16`	2.10	32.4	15.5	17	120	3.70
N, M grow	`rmat_p36`	4.72	73.8	15.6	38	274	3.71
N, M grow	`rmat_p72`	9.44	148.1	15.7	75	571	3.85
N, M grow	`rmat_p144`	18.87	297.3	15.8	151	1312	4.41
N, M grow	`rmat_p288`	37.75	596.4	15.8	302	3087	5.18
N, M grow	`rmat_p432`	56.62	895.5	15.8	453	4958	5.54
N, M grow	`rmat_p576`	75.50	1195.6	15.8	604	6866	5.74
N, M grow	`rmat_p720`	94.37	1495.6	15.8	755	9014	6.03
real	`web-Google`	0.88	5.1	5.8	7	23.5	4.61
real	`soc-LiveJournal1`	4.85	68.5	14.1	39	159	2.32
real	`com-friendster`	65.61	3612.1	55.1	525	24748	6.85

More edges on the same vertices. At $N = 2^{20}$ the time per iteration grows from 14 to 145 ms while M grows by a factor of 15. A least-squares line through the five graphs has a slope of 2.3 ns per edge and an intercept of 4 ns per vertex. The time is linear in M, and the cost of an edge, between 2.3 and 3.4 ns, does not grow with M: the vector x of these graphs occupies 8 MB and stays in the L3 cache however many edges read it.

More vertices and edges. From `rmat_p1` to `rmat_p720` the number of edges grows by a factor of 770 and the time by a factor of 1984, from 4.5 ms to 9.0 s. The time per edge rises from 2.3 to 6.0 ns. The number of edges per vertex is the same in all these graphs, so the increase comes from N: the vector x grows from 1 MB to 755 MB. It outgrows the 2 MB L2 cache of the core at $N = 0.26$ million and the 67.5 MB L3 cache of the socket at $N = 8.4$ million, and a growing share

of the reads of x is served from main memory.

Real graphs. The real graphs fit the same picture. soc-LiveJournal1, whose x occupies 39 MB, runs at 2.3 ns per edge. com-friendster, whose x occupies 525 MB, runs at 6.9 ns per edge and needs 24.7 s per iteration.

The serial run time is thus $T \approx t_e(N)\, M + t_v\, N$: linear in the number of edges, with a cost per edge t_e that the number of vertices raises from about 2.3 ns, when x fits in the cache, to 6–7 ns, when it does not. The same effect appears in the parallel results. A large graph has a slow one-core baseline per edge, and dividing it among more cores also divides x among more caches.

7 MPI scaling

7.1 Strong scaling

The two large real graphs are run on 1 to 720 cores with the edge partition and the ghost exchange (Figures 3 and 4, Table 5). The baseline T_1 is the MPI+OpenMP program on one core. It is within 5% of the serial program on both graphs (166 against 159 ms, and 25.3 against 24.7 s).

Pure MPI. On one node, pure MPI reaches a speedup of 16.3 on 72 cores for soc-LiveJournal1 ($E = 0.23$) and of 32.8 for com-friendster ($E = 0.46$). It is fastest on 144 cores for both graphs: 9.84 ms for soc-LiveJournal1 ($S = 17$) and 745 ms for com-friendster ($S = 34$). With more ranks the time rises again. soc-LiveJournal1 needs 34.22 ms on 288 ranks and 317 to 446 ms on 432 to 720 ranks, which is slower than on one core. com-friendster needs 2215 ms on 720 ranks, a speedup of 11.4 and an efficiency of 0.016. The MPI-only program therefore scales to two nodes on this cluster. Section 7.3 shows that the limit is the exchange between the ranks and not the computation.

MPI+OpenMP. The hybrid layout is faster than pure MPI at every core count above one. Up to 36 cores it is a single rank, and it scales almost ideally: the speedup on 36 cores is 35.1 for com-friendster ($E = 0.97$) and 27.3 for soc-LiveJournal1 ($E = 0.76$). For com-friendster the efficiency is even slightly above one from 2 to 16 cores (1.02 to 1.06); every additional core brings its own 2 MB of L2 cache. Beyond one node, the two graphs behave differently.

- *com-friendster* keeps scaling: 421 ms on one node, 278 ms on two, 193 ms on four, and 139 ms on 576 cores. This is a speedup of 183 over one core ($E = 0.32$) and of 3.0 over one node. On 720 cores the time is 162 ms.

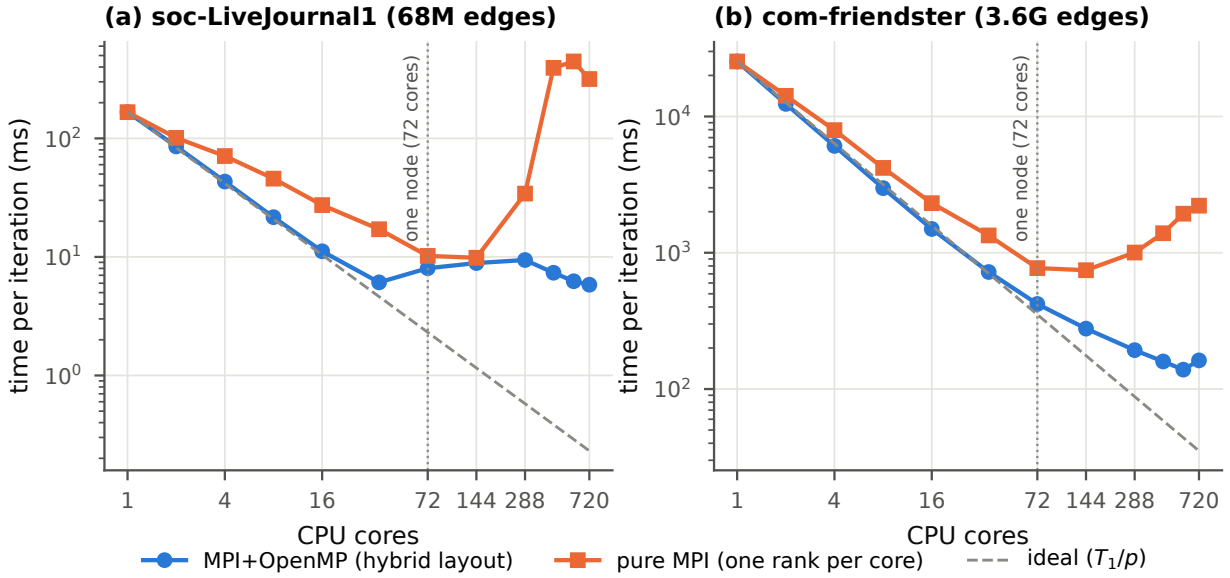


Figure 3. Strong scaling from 1 to 720 cores (10 nodes): time per iteration of the MPI+OpenMP program in the pure-MPI layout and in the hybrid layout.

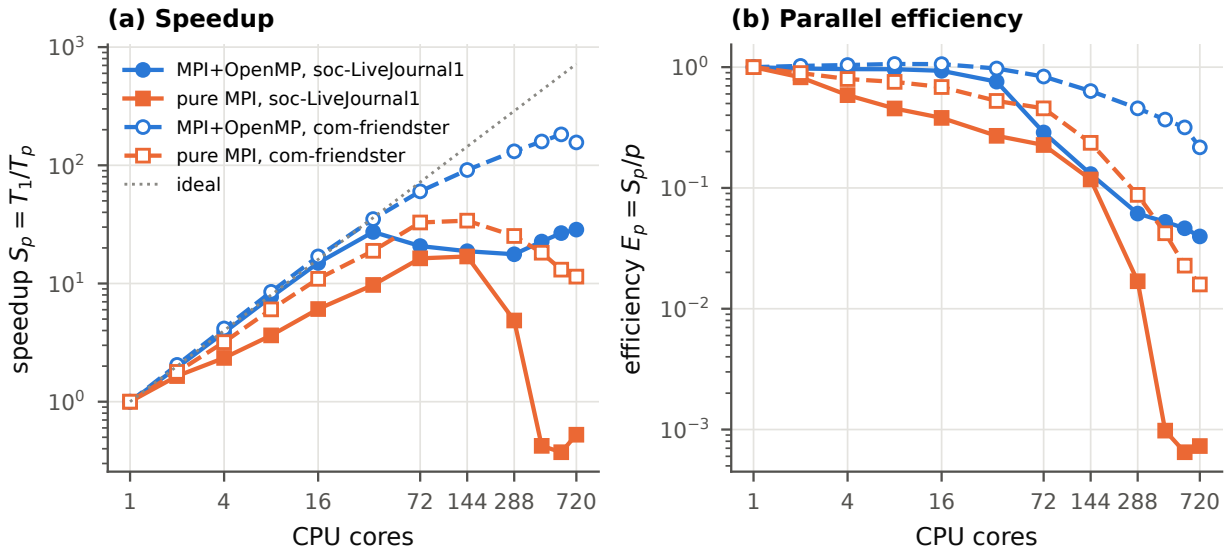


Figure 4. Speedup and parallel efficiency of the runs in Figure 3. Filled markers: soc-LiveJournal1; open markers: com-friendster.

Table 5. Strong scaling: time per iteration T_p (ms), speedup S_p and efficiency E_p. Up to 72 cores the runs use one node; 144 to 720 cores are 2 to 10 nodes.

	soc-LiveJournal1						com-friendster					
	hybrid			pure MPI			hybrid			pure MPI		
cores	T_p	S_p	E_p	T_p	S_p	E_p	T_p	S_p	E_p	T_p	S_p	E_p
1	166	1.0	1.00	166	1.0	1.00	25335	1.0	1.00	25335	1.0	1.00
2	85.5	1.9	0.97	101	1.6	0.82	12370	2.0	1.02	14181	1.8	0.89
4	43.2	3.9	0.96	71.0	2.3	0.59	6084	4.2	1.04	7940	3.2	0.80
8	21.6	7.7	0.96	45.7	3.6	0.45	2980	8.5	1.06	4194	6.0	0.76
16	11.2	15	0.93	27.4	6.1	0.38	1495	17	1.06	2313	11	0.68
36	6.10	27	0.76	17.1	9.7	0.27	722	35	0.97	1340	19	0.53
72	8.03	21	0.29	10.2	16	0.23	421	60	0.84	772	33	0.46
144	8.88	19	0.13	9.84	17	0.12	278	91	0.63	745	34	0.24
288	9.42	18	0.061	34.2	4.9	0.017	193	131	0.46	1004	25	0.088
432	7.35	23	0.052	394	0.4	0.00098	159	159	0.37	1391	18	0.042
576	6.24	27	0.046	446	0.4	0.00065	139	183	0.32	1934	13	0.023
720	5.83	29	0.04	317	0.5	0.00073	162	156	0.22	2215	11	0.016

- *soc-LiveJournal1* stops at one socket. One iteration takes 6.10 ms on 36 cores, which is less than one exchange of ghost values between nodes costs. From 72 to 720 cores the time stays between 5.8 and 9.4 ms; the best value, 5.83 ms on 720 cores, is a speedup of 29, against 27.3 on 36 cores.

On several nodes the hybrid layout is up to 14 times faster than pure MPI for com-friendster and up to 71 times for soc-LiveJournal1 (both on 576 cores).

7.2 Weak scaling

For weak scaling, `rmat_p`k is run on k cores, so the numbers of vertices and edges grow in proportion to the core count (Figure 5, Table 6). Every core has between 1.94 and 2.08 million edges; the number rises slightly with k, because R-MAT produces fewer duplicate edges in larger graphs. With ideal weak scaling the time per iteration would stay at $T_1 = 2.76$ ms, the one-core time of the MPI+OpenMP program on `rmat_p1`. (The unpinned serial program needs 4.5 ms on this graph.)

The time of the hybrid layout grows from 2.76 ms to 73.8 ms on 720 cores, a weak-scaling efficiency of 0.037. Pure MPI follows the hybrid layout up to 144 cores (38.6 against 23.9 ms), jumps to 754 ms on 288 ranks and reaches 2161 ms on 720 ranks. The columns t_{comp} and t_{comm} of

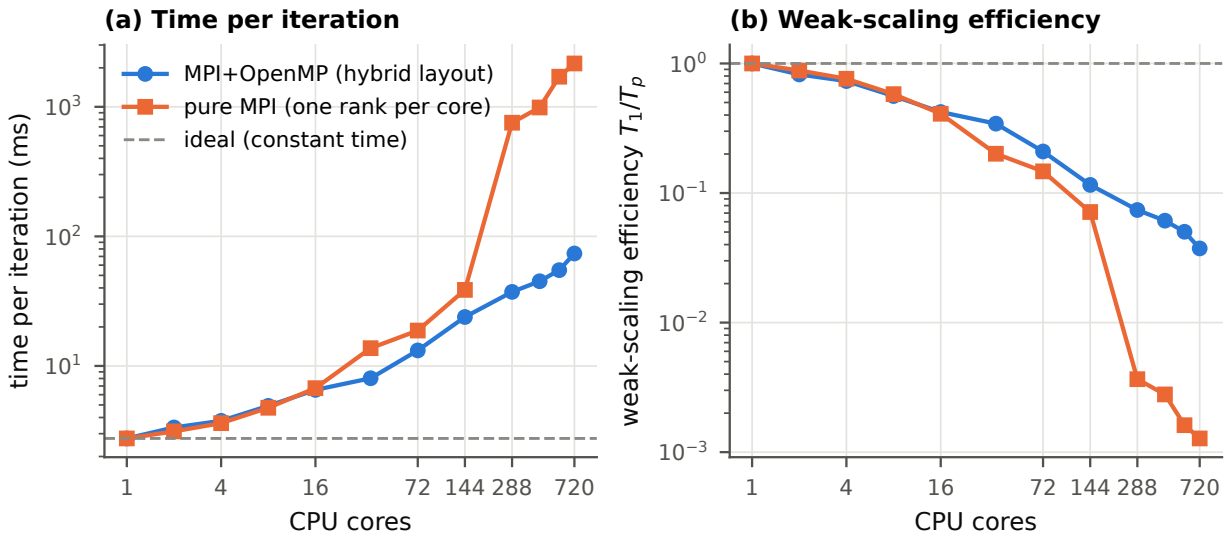


Figure 5. Weak scaling: `rmat_p`k on k cores, about 2 million edges per core, from 1.9 million edges on one core to 1.5 billion edges on 720 cores.

Table 6. Weak scaling on `rmat_p`k. M/p is the number of edges per core after duplicate removal. Times in ms per iteration.

				hybrid				pure MPI		
cores	N (10^6)	M (10^6)	M/p (10^6)	T_p	t_{comp}	t_{comm}	T_1/T_p	T_p	t_{comm}	T_1/T_p
1	0.13	2	1.94	2.76	2.76	–	1.00	2.76	–	1.00
2	0.26	4	1.97	3.35	3.35	–	0.82	3.12	0.17	0.88
4	0.52	8	1.99	3.77	3.77	–	0.73	3.62	0.36	0.76
8	1.05	16	2.01	4.92	4.92	–	0.56	4.76	0.73	0.58
16	2.10	32	2.03	6.53	6.53	–	0.42	6.74	1.39	0.41
36	4.72	74	2.05	8.03	8.03	–	0.34	13.7	3.70	0.20
72	9.44	148	2.06	13.2	8.42	4.79	0.21	18.8	6.58	0.15
144	18.87	297	2.06	23.9	11.1	13.1	0.12	38.6	23.9	0.071
288	37.75	596	2.07	37.3	14.6	23.3	0.074	754	737	0.0037
432	56.62	895	2.07	45.1	15.2	33.5	0.061	988	972	0.0028
576	75.50	1196	2.08	54.9	15.9	42.9	0.05	1707	1691	0.0016
720	94.37	1496	2.08	73.8	16.2	61.0	0.037	2161	2145	0.0013

Table 6 separate two causes.

- *Within one node* (1 to 72 cores) the time grows from 2.76 to 13.2 ms, an efficiency of 0.21, although no message leaves the node. The computation itself becomes slower, from 2.76 to 8.4 ms, for the two reasons seen so far. The graph outgrows the caches: x occupies 1 MB for `rmat_p1` and 75 MB for `rmat_p72`, and the serial time per edge on the same graphs rises from 2.3 to 3.9 ns (Table 4). And the cores share the memory bandwidth of the node: the STREAM triad delivers 18 GB/s to one core but 424 GB/s to 72 cores, 5.9 GB/s each (Section 8.1).
- *On several nodes* (144 to 720 cores) the computation stays between 11.1 and 16.2 ms, as intended with a constant number of edges per core, but the communication grows from 13.1 to 61.0 ms. The in-edges of a rank come from all over the graph, so its number of ghost vertices grows with the graph even though its number of edges is fixed.

Compared with the serial program on the same 1.5-billion-edge graph, the 720-core run is 122 times faster (9.0 s against 73.8 ms).

7.3 Impact of communication

An iteration makes three MPI calls. The two `MPI_Allreduce` calls carry one number each. The ghost exchange carries all the data. Figure 6 and Table 7 show the time spent in computation and

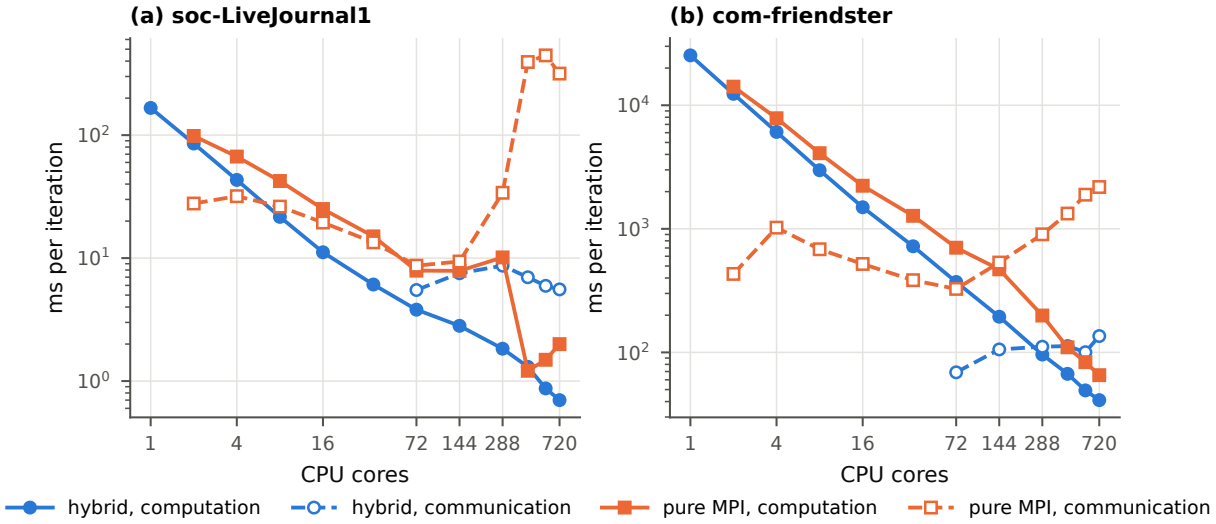


Figure 6. Computation time t_{comp} (filled markers) and communication time t_{comm} (open markers) per iteration in the strong-scaling runs. The hybrid layout has a single rank, and no communication, up to 36 cores.

Table 7. Strong-scaling runs on one to ten nodes: computation time t_{comp} and communication time t_{comm} per iteration (ms), and ghost data V received per iteration, summed over all ranks (MB).

	soc-LiveJournal1						com-friendster					
	hybrid			pure MPI			hybrid			pure MPI		
cores	t_{comp}	t_{comm}	V	t_{comp}	t_{comm}	V	t_{comp}	t_{comm}	V	t_{comp}	t_{comm}	V
72	3.81	5.50	24	7.91	8.68	229	371	69.0	324	703	326	8259
144	2.81	7.53	57	7.88	9.38	263	194	106	862	470	534	11699
288	1.83	8.68	97	10.1	34.0	291	96.1	111	1722	199	903	15450
432	1.31	6.99	123	1.21	393	306	67.0	113	2433	110	1330	17614
576	0.87	5.95	141	1.49	445	315	49.2	100	3057	83.4	1889	19079
720	0.70	5.57	157	2.00	317	322	41.1	136	3617	65.4	2180	20154

in MPI calls separately.

The computation time falls in proportion to the number of cores in both layouts. For com-friendster in the hybrid layout it is 371 ms on 72 cores and 41 ms on 720. The communication time does not fall: it is 69 ms on 72 cores and 136 ms on 720. From 288 cores on, an iteration spends more time in MPI calls than in computation (111 against 96 ms), and this is where the efficiency falls below 0.5. For soc-LiveJournal1 the communication exceeds the computation as soon as there is more than one rank (5.50 against 3.81 ms on 72 cores). Both the volume of the exchange and its number of messages matter, in different layouts.

Volume. The ghost data per iteration grows with the number of ranks (column V), because a value that several ranks need is sent to each of them. At the same core count the hybrid layout moves 2 to 10 times less data than pure MPI for soc-LiveJournal1, and 6 to 25 times less for com-friendster: the 36 threads of a rank share one copy of each ghost value, whereas 36 separate ranks each receive their own. In the hybrid layout the time of the exchange follows this volume. A rank receives between 181 and 215 MB per iteration for com-friendster, and between 7.8 and 14.2 MB for soc-LiveJournal1, at a rate of 1.3 to 2.4 GB/s in all multi-rank runs of both graphs. The volume per rank hardly changes with the number of ranks: a rank that owns 5% of the edges of com-friendster still needs the values of 34% of all vertices from other ranks. The communication time per iteration of com-friendster therefore stays near 100 ms while the computation time shrinks, and this bounds the speedup.

Number of messages. In pure MPI, the volume per rank is small. On 720 ranks a rank receives 28 MB per iteration for com-friendster, yet the exchange takes 2180 ms, a rate of 13 MB/s, more than a hundred times below the rate of the hybrid layout. Here the cost follows the number of messages. Almost every rank has ghost vertices on almost every other rank, so one exchange consists of up to $P(P-1)$ messages: 380 for the 20 ranks of the hybrid layout on ten nodes, 82,656 for 288 single-threaded ranks and 517,680 for 720. Between nodes, every pair of ranks communicates over its own TCP connection, and every message passes through the network stack of the operating system on both sides. On 576 ranks a node holds $72 \times 504 \approx 36{,}000$ connections. The run times in this regime also vary from run to run: the same configuration, soc-LiveJournal1 on 288 ranks, was launched in 5 different experiments and took between 34 and 156 ms per iteration.

Waiting and moving data. t_{comm} contains the time a rank waits inside an MPI call for the others. To separate the two, the hybrid layout was also run with the option `--split`, which puts a barrier in front of every MPI call. For com-friendster the barriers take 13 to 22 ms per iteration at every core count, and the remaining communication time, which is pure data movement, grows from 50 ms on 72 cores to 105 ms on 720. The ranks finish their computation almost together, as the edge partition intends, and the time goes into moving the ghost values.

Within one node. Communication is costly even through shared memory. Going from one rank with 36 threads to two ranks with 36 threads each makes soc-LiveJournal1 slower, 6.10 ms against 8.03 ms: the second rank reduces the computation to 3.78 ms but adds an exchange of 4.19 ms.

7.4 Partition and exchange method

Table 8. The four combinations of partition and exchange method on 288 ranks (pure MPI, four nodes). Times in ms per iteration; V is the data received per iteration, summed over all ranks.

graph	partition	exchange	T_p	t_{comp}	t_{comm}	imbalance	V (MB)
`soc-LiveJournal1`	vertex	allgather	48.8	5.53	48.4	8.62	11,130
`soc-LiveJournal1`	vertex	ghost	24.8	4.38	24.6	8.62	256
`soc-LiveJournal1`	edge	allgather	57.0	2.39	56.7	1.05	11,130
`soc-LiveJournal1`	edge	ghost	47.7	5.27	47.5	1.05	291
`com-friendster`	vertex	allgather	565	234	547	1.79	150,637
`com-friendster`	vertex	ghost	1013	262	995	1.79	15,045
`com-friendster`	edge	allgather	682	145	596	1.00	150,637
`com-friendster`	edge	ghost	1045	194	944	1.00	15,450

Table 8 compares the two partitions and the two exchange methods on 288 ranks.

The edge partition removes the load imbalance. With equal numbers of vertices, the most loaded rank has 8.62 times the average number of in-edges for soc-LiveJournal1 and 1.79 times for com-friendster; with the edge partition the values are 1.05 and 1.00. For com-friendster, the computation time of the slowest rank falls from 262 to 194 ms.

The ghost exchange reduces the volume by a factor of 38 for soc-LiveJournal1, from 11 GB to 291 MB per iteration, and by a factor of 10 for com-friendster. On 288 single-threaded ranks, however, the time follows the number of messages and not the volume (Section 7.3). `MPI_Allgatherv` moves the larger volume with a schedule in which each rank exchanges data with a few partners only, and for com-friendster it is faster than the ghost exchange (682 against 1045 ms). For soc-LiveJournal1 the total times on 288 ranks vary too much between runs to rank the four combinations (Section 7.3). With few ranks the lower volume decides. In the validation runs on web-Google with 4 ranks of 4 threads, the ghost exchange is 15% faster than the allgather (2.57 against 3.04 ms per iteration with the edge partition), and on the 20 ranks of the hybrid layout an allgather would deliver about 500 MB of com-friendster to every rank where the ghost exchange delivers 181 MB. All scaling runs therefore use the edge partition with the ghost exchange.

8 MPI+X

8.1 X = OpenMP

Threads and memory bandwidth. Figure 7a compares the effective bandwidth B of the iteration with the STREAM triad on one node. For com-friendster, B grows from 1.8 GB/s on one core to 64 GB/s on 36 cores and 110 GB/s on the full node, which is 26% of the 424 GB/s that the triad reaches. This is close to what the memory of the node can deliver for this access pattern: if every read of x loads a 64-byte cache line, an iteration on com-friendster transfers about five times the bytes it uses, so B can reach about a fifth of the bandwidth for sequential access. For soc-LiveJournal1, whose x fits in the L3 cache, B reaches 170 GB/s on 36 cores, more than half of the triad (316 GB/s). OpenMP threads therefore speed up the iteration only until the memory bandwidth of the node is used up, which shows in the efficiency of 0.76 for soc-LiveJournal1 on 36 cores and in the loss of weak-scaling efficiency within one node (Section 7.2).

Ranks and threads on one node. Figure 7b divides the 72 cores of a node into 1 to 72 ranks. For com-friendster, two ranks with 36 threads are fastest (421 ms). One rank with 72 threads is 1.5 times slower (634 ms): a rank reads and allocates its arrays on its master thread, so its data resides in the memory of one NUMA domain, and with 72 threads half of them work on the memory of the other socket. With more than two ranks, the communication grows, up to 772 ms for 72 single-threaded ranks. For soc-LiveJournal1 the order is different. One rank with 72 threads is fastest (5.07 ms): its x fits in the L3 cache of either socket, so memory placement matters little, and a single rank needs no exchange at all. Two ranks need 7.99 ms and 72 ranks 10.25 ms.

Ranks and threads on four nodes. On 288 cores (Figure 7c) the fastest layouts are 8 ranks of 36 threads for com-friendster (194 ms) and 16 ranks of 18 threads for soc-LiveJournal1 (6.61 ms), that is, one or two ranks per NUMA domain. Four ranks of 72 threads are 1.6 times slower for both graphs, again because of memory placement. More ranks mean more ghost data and more messages: 72 ranks of 4 threads are 2.5 to 2.7 times slower than the best layout, and 288 single-threaded ranks are 13 times slower for soc-LiveJournal1 and 5.4 times for com-friendster. One rank per NUMA domain keeps the data of a rank in local memory and uses the smallest number of ranks that does so, which is why it is the layout of the hybrid runs.

8.2 X = CUDA on one GPU

Kernels. Table 9 and Figure 8a give the time per iteration on soc-LiveJournal1 for every kernel. In the single-GPU program the thread-per-row kernel is the fastest of our kernels (5.42 ms), followed

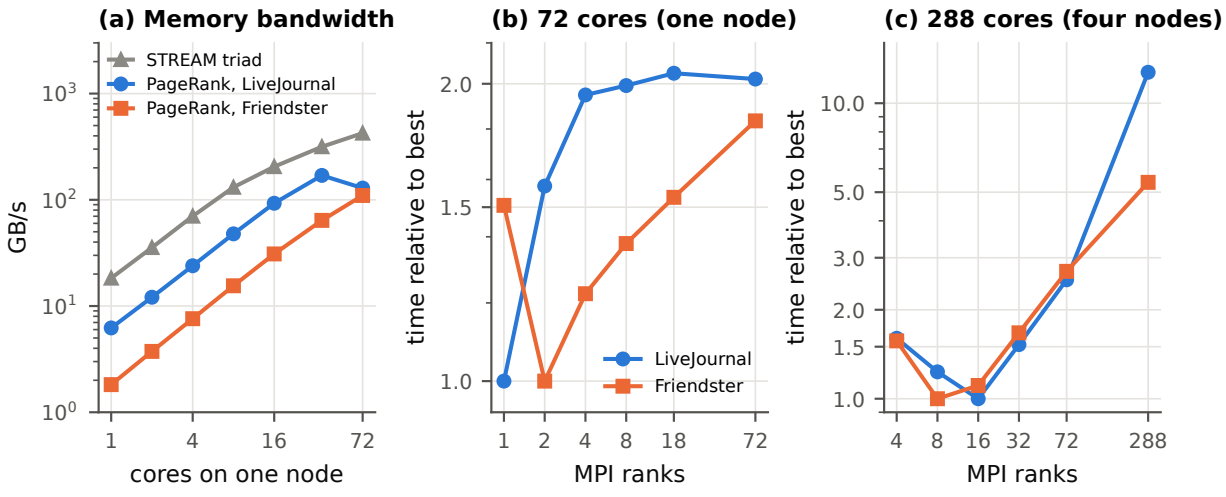


Figure 7. (a) Memory bandwidth on one node: STREAM triad, and effective bandwidth of the PageRank iteration in the hybrid layout. (b), (c) The same number of cores divided into different numbers of ranks; every rank has cores/ranks threads.

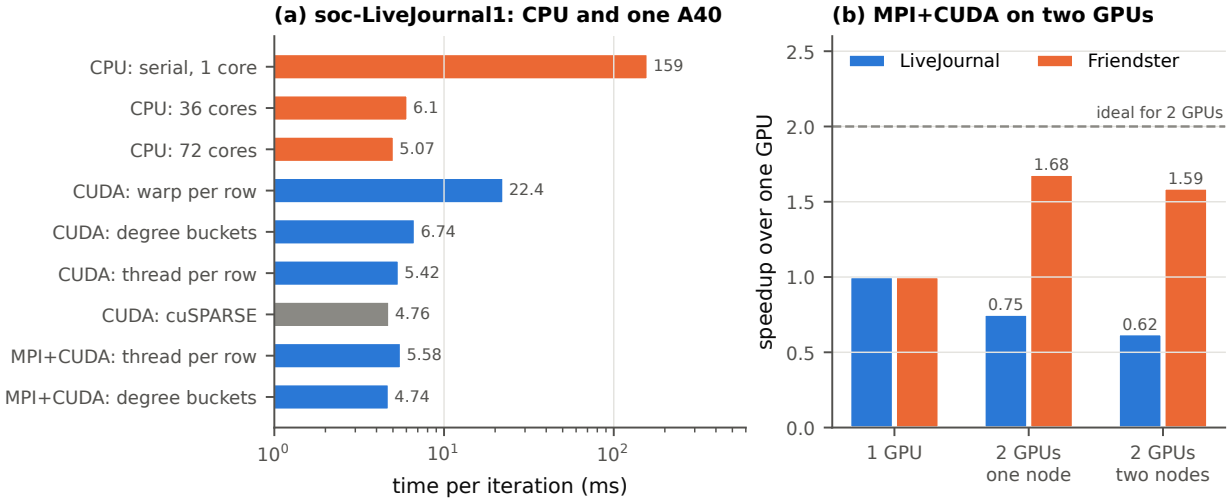


Figure 8. (a) Time per iteration on soc-LiveJournal1 for the CPU (hybrid layout with a single rank) and for one A40 GPU. (b) Speedup of the MPI+CUDA program with two GPUs over one GPU.

Table 9. SpMV kernels on one A40 GPU, soc-LiveJournal1: time per iteration and effective bandwidth B.

program	SpMV kernel	ms / iteration	B (GB/s)
CUDA	one thread per row	5.42	187
CUDA	one warp per row	22.40	45
CUDA	degree buckets	6.74	151
CUDA	`cusparseSpMV` (reference)	4.76	213
MPI+CUDA, 1 GPU	one thread per row	5.58	185
MPI+CUDA, 1 GPU	degree buckets	4.74	219

by the degree buckets (6.74 ms). The warp-per-row kernel is 4 times slower than the thread-per-row kernel: a row of this graph has 14 in-edges on average, so most of the 32 threads of a warp have no edge to add. In the MPI+CUDA program on one GPU, the degree-bucket kernel takes 4.74 ms and the thread-per-row kernel 5.58 ms. Our fastest kernel thus matches `cusparseSpMV` (4.76 ms).

GPU and CPU. On soc-LiveJournal1, one A40 is 33 times faster than the serial program, 1.3 times faster than 36 cores (6.10 ms) and about as fast as the full 72-core node (5.07 ms). com-friendster has more than 2^{31} edges and runs in the MPI+CUDA program, where one GPU needs 584 ms per iteration. This is 1.4 times slower than one CPU node (421 ms).

Why the GPU is not faster. The iteration is limited by memory access on both platforms, and their memory systems are comparable. We measured the memory of the A40 with a probe (`tools/gpu_bw.cu`) on 2 GB arrays. A copy within GPU memory reaches 565 GB/s and the STREAM triad 542 GB/s, only 1.3 times the 424 GB/s of the triad on a CPU node. A random gather $y_i = x_{\mathrm{idx}_i}$, the access pattern of the SpMV, reaches 113 GB/s, 4.8 times less than the triad. The PageRank kernels have an effective bandwidth of 187 to 219 GB/s on soc-LiveJournal1 and of 79 GB/s on com-friendster, against 170 GB/s on one CPU socket for soc-LiveJournal1 and 110 GB/s on one CPU node for com-friendster. The GPU has many more arithmetic units than the CPU, but the iteration performs one addition per 12 bytes of data and cannot use them. The two platforms move a similar number of bytes per second for this access pattern, and their run times differ by less than a factor of 1.5 in either direction.

8.3 X = CUDA on two GPUs

Figure 8b shows the MPI+CUDA program on two GPUs, either on the same node or on two different nodes.

For com-friendster, two GPUs on one node need 348 ms per iteration instead of 584 ms, a speedup of 1.68 and an efficiency of 0.84. With one GPU on each of two nodes the time is 368 ms, a

speedup of 1.59. The computation is halved, from 584 to 297 ms per GPU. The exchange of 324 MB of ghost values adds 48 ms of MPI time on one node, or 68 ms over TCP between two nodes, and 13 ms for copying the buffers between GPU and host.

For soc-LiveJournal1, one GPU needs 4.74 ms. Two GPUs reduce the computation to 3.08 ms, but the exchange costs 3.56 ms on one node and 4.88 ms between two nodes, plus 0.99 ms of copies. The iteration takes 6.32 and 7.63 ms and is slower than on one GPU. This is the same rule as on the CPU: a second device pays off when the computation it takes over is larger than the exchange it adds.

9 Discussion

9.1 Conclusions about the data

web-Google. The iteration converges in 118 iterations. The residual falls by a factor that settles at 0.848 per iteration (Figure 9a), just below the bound $d = 0.85$ of Section 2. The resulting ranks are very unequal. The top 0.1% of the pages hold 11.4% of the total rank, the top 1% hold 27.1% and the top 10% hold 56.8%. The highest rank is 809 times the uniform value $1/N$. The 161,168 pages without in-links, 18% of all pages, share the lowest rank, 0.22 times the uniform value.

The rank of a page grows steadily with its number of in-links (Figure 9b); the Spearman correlation between PageRank and in-degree is 0.85. The in-degree does not determine the rank, however. Of the 100 highest-ranked pages, 88 are also among the 100 pages with the most in-links, and the page with the second-highest rank has only the 12th largest in-degree. A page ranks high when the pages that link to it rank high themselves.

soc-LiveJournal1 and com-friendster. For the two large graphs, the measurements lead to four conclusions about their structure.

- The edges are concentrated on few vertices and in parts of the identifier range. The largest in-degree of soc-LiveJournal1 is 13,905 against an average of 14, and one of 288 blocks with equal numbers of vertices holds 8.62 times the average number of in-edges. com-friendster is more even (1.79 times).
- The vertex numbering has little locality. A contiguous block with 5% of the edges needs the values of 34% of all vertices from outside the block in com-friendster, and of 20% in soc-LiveJournal1. A partition into contiguous blocks therefore cannot keep most edges inside the blocks.

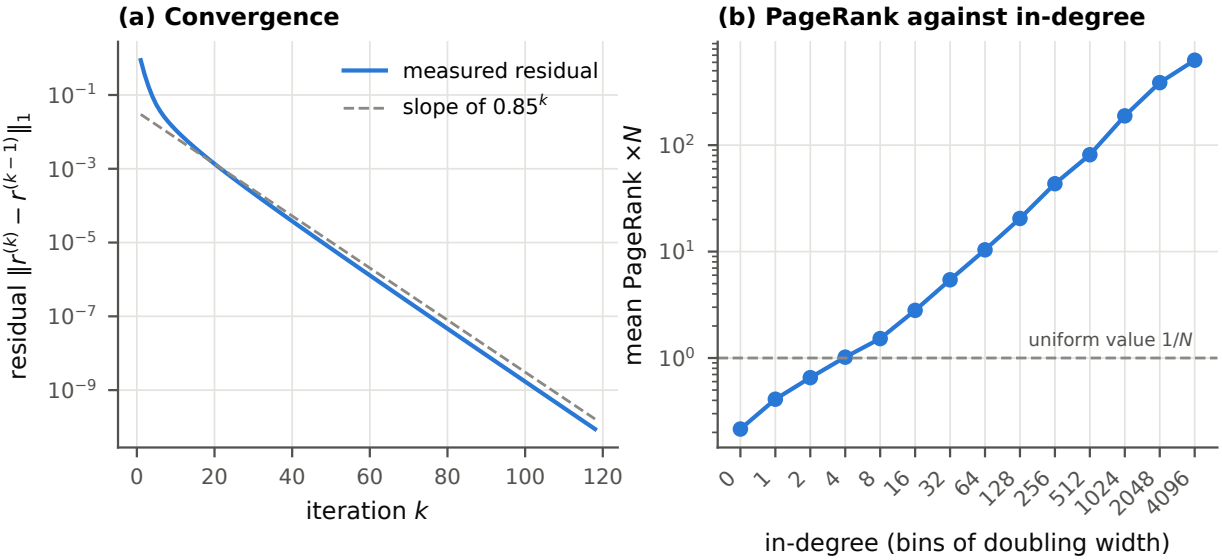


Figure 9. web-Google. (a) Residual of the serial program in every iteration. (b) Mean PageRank of the vertices in each in-degree class, relative to the uniform value $1/N$.

- The size of the vertex set relative to the cache sets the cost of an edge. soc-LiveJournal1 (39 MB of x) costs 2.3 ns per edge in the serial program and com-friendster (525 MB) costs 6.9 ns.
- The amount of local work per ghost value decides how far a graph scales. com-friendster has 55 edges per vertex, and on 144 cores a rank processes 34 edges for every ghost value it receives. soc-LiveJournal1 has 14 edges per vertex and processes 10. The first graph scales to 576 cores and the second to one socket.

9.2 How the problem was parallelised

The parallelisation is data-parallel throughout: the vertices and their in-edges are divided, and every unit applies the same iteration to its part. Different techniques serve different parts of the problem.

- *Between nodes and between NUMA domains: MPI.* The rows of the matrix are divided into contiguous blocks with equal numbers of edges, so every rank has the same amount of SpMV work, and every rank keeps its block in the memory next to its cores. The ghost exchange delivers exactly the values a rank needs.
- *Within a NUMA domain: OpenMP.* The loops over the vertices of a rank are shared among threads, with a dynamic schedule for the SpMV because the rows differ in length. Threads share the arrays of the rank, so cores of the same socket exchange no messages and hold one copy of each ghost value.
- *On GPUs: CUDA.* Every row is processed by its own GPU thread, or by a warp or a block if it is long. All arrays stay on the GPU, and MPI connects several GPUs in the same way as it connects NUMA domains.
- *Global sums.* The dangling mass and the residual are reductions at every level: OpenMP reductions within a rank, block reductions on the GPU, and `MPI_Allreduce` across ranks.
- *Input.* Every rank reads its own slice of the graph file, so loading needs no communication beyond the broadcast of the block boundaries.

9.3 Which parts need the most communication

The ghost exchange needs by far the most communication. It carries all the data that moves between ranks, up to 3.6 GB per iteration for com-friendster in the hybrid layout and 20 GB in pure MPI, in up to $P(P-1)$ messages. On several nodes it takes more time than the computation for soc-LiveJournal1 at every core count and for com-friendster from 288 cores on. The two reductions carry one number each; what they cost is that all ranks wait for the slowest one, 13 to 22 ms per iteration for com-friendster in the hybrid layout. The setup communicates once: the block boundaries are broadcast and every rank sends its list of ghost vertices to their owners. In the MPI+CUDA program, the ghost values additionally cross between GPU and host memory in both directions, which costs a fifth to a quarter of the MPI time.

9.4 What limits the scaling

On one node the limit is memory bandwidth, and on several nodes it is the ghost exchange. The exchange takes about the same time per iteration however many ranks share the work, because every rank refers to a large part of the graph. Two changes would move this limit. Renumbering the vertices with a graph partitioner, so that most edges stay inside a block, would reduce both the number of ghost values and the number of ranks a rank communicates with; this addresses the volume in the hybrid layout and the message count in pure MPI. An MPI library with RDMA support would lower the cost of each message, which matters most for pure MPI, and a CUDA-aware MPI would remove the copies through host memory in the MPI+CUDA program.

9.5 What the results mean for applications

The measurements answer the practical question of how much hardware a PageRank computation of a given size should use. A graph of the size of soc-LiveJournal1, tens of millions of edges, is

ranked fastest on a single 72-core node or a single GPU; distributing it over more nodes or GPUs only adds communication. A graph of the size of com-friendster, billions of edges, benefits from a cluster: one iteration takes 139 ms on 576 cores instead of 25.3 s on one core, so a run of 147 iterations, the most that a tolerance of 10^{-10} can require, takes about 20 seconds instead of about an hour, and the ranks of a changing graph of this size can be recomputed that much more often. The layout matters as much as the core count: with one MPI process per core the same cluster is 14 times slower than with one process per NUMA domain and OpenMP threads inside it.

The results carry over to other methods, because the kernel is a sparse matrix–vector product on a power-law graph. Personalised PageRank for recommendations, label propagation and other iterative graph algorithms repeat the same step and meet the same two limits, memory bandwidth within a node and the exchange of vertex values between nodes.

AI usage

We used AI coding assistants (Claude and Codex) for coding, for drafting this report, and for pushing the report to the `submission` branch of the repository. All timings and validation results are the output of our programs on Vanda.

Source code

The code is in a public GitHub repository:

`https://github.com/yespiggy/Project3/tree/submission`

The repository contains the four programs (`src/`), the preprocessing tool, the graph generator, the bandwidth probes and the validation scripts (`tools/`), the PBS job scripts of every experiment (`jobs/final/`), the raw result files (`results_final/`) and `tools/make_report.py`, the script that produces the figures and tables of this report. `README.md` explains how to build the programs, obtain the data, run the experiments and rebuild the report.

References

[1] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank citation ranking: bringing order to the web. Technical report, Stanford InfoLab, 1999.

[2] D. F. Gleich. PageRank beyond the Web. SIAM Review 57(3), 2015.

[3] J. Leskovec, A. Krevl. SNAP datasets: Stanford large network dataset collection. `http://snap.stanford.edu/data`, 2014.

[4] D. Chakrabarti, Y. Zhan, C. Faloutsos. R-MAT: a recursive model for graph mining. SIAM International Conference on Data Mining, 2004.

[5] Graph500 benchmark specification. `https://graph500.org`.

[6] J. D. McCalpin. Memory bandwidth and machine balance in current high performance computers. IEEE TCCA Newsletter, 1995.